

A Hierarchical Edge Control Plane for Policy-Aware Multi-Module AI Orchestration

Dr. S. Suresh Kumar / ScholarMaster Research Group
Swarnandhra College of Engineering & Technology (Autonomous)

Abstract—Edge deployments of multi-model AI perception pipelines face orchestration challenges that are largely independent of individual model accuracy. Dynamic coordination of heterogeneous inference pipelines subject to shared latency, computational, and policy constraints presents a control theory challenge. The tight coupling of such systems can create cascading fault conditions in the perception pipeline while introducing unacceptable response time delays in the cloud-coordination approach for instructional contexts. To deal with this orchestration problem, this work proposes a hierarchical control-plane abstraction for policy-aware coordination of multi-module AI workloads at the edge. The paper describes a layered execution environment, including Perception, Reasoning, and Governance, with limited cross-layer communication via typed, bounded context vectors. It introduces the Compute Justification Model for Inference Rate Governance (IRG), which allows for dynamic activation/suppression of sensing modules based on lecture phase and associated heuristics in order to manage computational cost. System resilience is evaluated via fault injection that exposes particular orchestrator to deterministic failure scenarios including module timeouts, memory pressure, and routing perturbations. To assess orchestration behavior independently from downstream model accuracy, we introduce four policy-aware metrics: Inference Suppression Ratio, Ethical Compute Utilization, Context-Aware Duty Cycle, and Layer Isolation Factor. The experiments demonstrate that orchestration according to hierarchical policies can localize failures and enforce operating policies down to individual model layers.

Keywords—Hierarchical Orchestration, Policy-Aware Control Plane, Inference Rate Governance, Deterministic Degradation, Fault Containment, Edge-Native Activation.

\begin{abstract}

Edge deployments of multi-model AI perception pipelines face orchestration challenges that are largely independent of individual model accuracy. Dynamic coordination of heterogeneous inference pipelines subject to shared latency, computational, and policy constraints presents a control theory challenge. The tight coupling of such systems can create cascading fault conditions in the perception pipeline while introducing unacceptable response time delays in the cloud-coordination approach for instructional contexts.

To deal with this orchestration problem, this work proposes a hierarchical control-plane abstraction for policy-aware coordination of multi-module AI workloads at the edge. The paper describes a layered execution environment, including Perception, Reasoning, and Governance, with limited cross-layer communication via typed, bounded context vectors. It introduces the Compute Justification

Model for Inference Rate Governance (IRG), which allows for dynamic activation/suppression of sensing modules based on lecture phase and associated heuristics in order to manage computational cost.

System resilience is evaluated via fault injection that exposes particular orchestrator to deterministic failure scenarios including module timeouts, memory pressure, and routing perturbations. To assess orchestration behavior independently from downstream model accuracy, we introduce four policy-aware metrics: Inference Suppression Ratio, Ethical Compute Utilization, Context-Aware Duty Cycle, and Layer Isolation Factor. The experiments demonstrate that orchestration according to hierarchical policies can localize failures and enforce operating policies down to individual model layers.

\end{abstract}

\begin{IEEEkeywords}

Hierarchical Orchestration, Policy-Aware Control Plane, Inference Rate Governance, Deterministic Degradation, Fault Containment, Edge-Native Activation.

\end{IEEEkeywords}

INTRODUCTION

The main idea of this paper is that hierarchical policy-aware orchestration can provide deterministic and fault-contained coordination of heterogeneous edge AI modules operated within common constraints. Edge intelligent systems often feature multiple domainspecific AI modules hosted at the edge, running concurrently, for instance, biometric recognition, pose estimation, and speech understanding. While each can be optimized individually, their joint orchestration in a shared runtime environment introduces a distinct coordination problem [Ref].

Architectural Fragility in Flat Multi-Model Pipelines

When modules are tightly coupled in flat execution pipelines, localized crashes in one heavy perception model can easily propagate, causing cascading failure across the entire sensing system. Shared compute contention interferes with other wise independent modules, while cloud mediated orchestration introduces unpredictable network latency that fails to meet the demands of interactive modeling [Ref]. Furthermore, naive always-on sensing waste computing resources by performing heavy duty processing when not needed. Orchestration destabilization is caused by topological factors rather than mere compute model inefficiency.

Contributions and Subsystem Scope

This work explores whether a logical control-plane abstraction can be leveraged to address these coordination challenges. We envision a layered execution paradigm, where the system is partitioned into three distinct spheres of influence: Perception, Reasoning, and Governance. By enforcing information and decision isolation within these domains, we propose the following contributions:

- A hierarchical control-plane abstraction relying on typed context vectors for orchestration isolation [Ref].
- A Compute Justification Model formalizing Inference Rate Governance (IRG) for policy-aligned module activation.
- Explicit failure containment strategies ensuring graceful degradation when individual modules fail [Ref].
- Validation via fault injection to confirm orchestration resilience under deterministic failure scenarios [Ref].

Within the ScholarMaster architecture, this subsystem takes care of the activation of other modules, controls their orchestration logic, and only these three things. It does not assess survivability envelopes (Paper 10), optimize flash memory durability (Paper 12), optimize federated adaptation (Paper 13), develop runtime verification theorems (Paper 18), or define irreversible governance doctrine (Paper 17).

RELATED WORK

Prior work on AI orchestration is dominated by cloud- centered approaches. For example, AWS IoT Hub offers cloud-based orchestration of distributed sensor systems [Ref], but such approaches typically incur unacceptable latencies due to the round-trip time to the cloud. Our approach keeps event stream processing on the edge in order to minimize control loop delays. Recent cloud-first reference architectures for edge analytics emphasize horizontal scalability but are typically unable to meet the latency constraints of time-critical classroom analysis.

In the commercial sector, affect recognition platforms typically utilize monolithic models that carry out end-to-end facial emotion recognition [Ref]. These systems are often resident on an edge device, but can have brittle failure modes, for instance, if the primary facial model crashes due to a faulty input frame or memory constraints, downstream processing is halted completely. By contrast, the hierarchical design allows the orchestrator to route logic to fallback modes when necessary—such as pose-only sensing—when heavy perception models fail [Ref]. Furthermore, while edge frameworks such as TensorFlow Lite perform well on optimizing single model inference [Ref], they tend to have limited support for coordination between modules, operational policies, and failure-aware scheduling.

Federated learning frameworks such as TensorFlow Federated [Ref] and PySyft aim to enable privacy-preserving model training at distributed nodes. They mainly focus on the aggregation of model weights during the training phase but leave the coordination of models at inference time, fault tolerance, and governance to higher level infrastructure. In this work, we contribute to the related but distinct problem of orchestrating pre-trained and heterogeneous models at inference time [Ref].

Recent evaluations have shown the feasibility of executing microservices at the edge for the continuous video analytics [Ref]. Standard orchestrators do not provide native primitives for the suppression of privacy-sensitive modules. The establishment of structured sensing boundaries in smart educational environments is widely acknowledged to be a necessity. The proposed framework performs this task by design within the orchestration layer using a Compute Justification Model that adheres to privacy-by-design principles [Ref].

PROBLEM STATEMENT

Coordinating heterogeneous AI modules inside a single edge deployment introduces several competing architectural requirements:

- **Boundary Confinement:** High-fidelity perception data (like raw video frames) must remain confined to lower abstraction layers to limit unnecessary exposure to upper-layer business logic [Ref].
- **Real-Time Responsiveness:** Cross-module coordination has to happen within tight latency envelopes to preserve systemic interactivity [Ref].
- **Failure Tolerance:** Individual module faults should gracefully degrade overall functionality rather than halting the entire sensing system [Ref].

Architectures that favours one of the dimension heavily may neglect others. The proposed hierarchical design attempt to alleviate constraint of these dimension by being explicit through layering and policy driven activation of modules.

The Compute Allocation Problem

One manifestation of this orchestration challenge is inefficient compute allocation during typical classroom activity. To illustrate the potential of such orchestration to be non-optimal in terms of computing resources, let us take a standard 60 minute class, broken down into a 10 minute attendance-taking segment, 30 minutes of passive video-based instruction during which the cognitive load is relatively low, a 15 minute question-and-answer segment with high cognitive load, and a 5 minute segment for dismissing the class.

A naive edge deployment might execute face detection, pose estimation, and ASR continuously at 30 frames per second for the full hour, consuming:

$$\text{Equation: } C_{\text{naive}} = 60 \times 60 \times 30 = 108,000 \text{ inference cycles}$$

However, heavy compute is actually only operationally justified during the attendance phase (10 minutes) and the active Q&A session (15 minutes). The justified cycle count is significantly lower:

$$\text{Equation: } C_{\text{justified}} = (10 + 15) \times 60 \times 30 = 45,000 \text{ inference cycles}$$

Consider that the naive algorithm would spend about 63000 cycles scanning the input where there is no valid data. Our algorithm effectively removes this overhead and only scans those parts of the input which are needed for processing according to the operational policy.

HIERARCHICAL CONTROL PLANE ABSTRACTION

To address the aforementioned orchestration constraints we propose to consider a layered execution environment that supervises module activation. The logical control abstraction is formalized as a supervisory automaton governing module activation. These logical divisions operate within the broader runtime architecture, maintaining strict logical separation from the underlying data plane.

Supervisory Automaton Formalization

Formally, let M denote the set of all available perception modules (e.g., vision, audio), and let $H: M \rightarrow \{\text{healthy, degraded, failed}\}$ indicate the health of each module. Let $P(t)$ and $B_{\text{remaining}}$ denote the policy chosen at time t and the computational budget remaining for the day, respectively.

We define a governance activation function that outputs the active module set $A(t) \subseteq M$:

$$\text{Equation: } A(t) = f(P(t), H(M), B_{\text{remaining}})$$

This establishes a formal state machine in which module activation is resolved deterministically based on the current policy phase, the hardware's health, and operational limits [Ref]. To prevent race-induced nondeterminism during asynchronous health updates, the function f is deterministic and side-effect free with respect to the perception modules, executed within a single-threaded governance loop.

The Logical Control Abstraction separates responsibilities vertically. Raw perception data is restricted entirely to the lowest logical boundary, ensuring that Governance only processes aggregated statistics.

Typed Context Vectors as Coordination Boundaries

Maintaining strict separation between architectural layers requires more than an interface; it requires a coordination-boundary mechanism. Communication across layers is restricted to typed and bounded context vectors:

$$\text{Equation: } \vec{C}_{\text{Perc to Reas}} = \{(\text{event_type, zone, timestamp})\}$$

By receiving only abstracted event tuples via $\vec{C}_{\text{Perc to Reas}}$, the Reasoning and Governance layers operate without direct exposure to raw sensory data. Typed vectors constrain semantic propagation so that upper layers receive bounded abstractions instead of high-fidelity signals. Orchestration isolation structurally depends on this restricted cross-layer visibility.

Layer Isolation Factor (LIF)

To assess the quality of orchestration topology, we define Layer Isolation Factor (LIF). Scored per each fault injection scenario, LIF quantifies isolation topology, not uptime probability. Given an isolated crash, does it cascade?

$$\text{Equation: } LIF = \frac{\text{Failures Contained in Origin Layer}}{\text{Total Module Failures}} \times 100\%$$

In our evaluations, LIF averaged 95%, demonstrating strong boundary confinement, though exact containment ratios naturally depend on the specific fault frequency of the underlying modules [Ref].

COMPUTE JUSTIFICATION MODEL

Continuous use of all perceptual modules rarely takes place in instruction, as the Compute Justification Model highlights by stating that allocations are made according to policy context rather than energy considerations alone. The Governance layer allows individual modules to operate according to a heuristic schedule defined for the type of lecture being given, thus making inference suppression contextually appropriate.

Scheduling Formalism and IRG

We formalize Inference Rate Governance (IRG) as an orchestration-aware compute restraint mechanism, rather than generic

power-saving logic. IRG governs unnecessary sensing activation, and this scheduling suppression directly brings down orchestration pressure. Let $C_{\text{sched}}(t)$ be the scheduled inference cycles at time t , for a continuously-running, full-session baseline. And let $J(t) \in \{0,1\}$ be a binary policy justification, indicating whether a phase requires active sensing (1) versus rest (0). The executed inference cycles $C_{\text{exec}}(t)$ are defined as:

$$\text{Equation: } C_{\text{exec}}(t) = C_{\text{sched}}(t) \cdot J(t)$$

Based on this abstraction, the Inference Suppression Ratio (ISR) quantifies the algorithmic compute savings:

$$\text{Equation: } ISR = 1 - \frac{\sum_t C_{\text{exec}}(t)}{\sum_t C_{\text{sched}}(t)}$$

ISR reflects the proportion of scheduled compute that is intentionally suppressed under policy control. While higher ISR values indicate reduced resource usage, their interpretation depends on alignment with institutional policy rather than raw suppression magnitude.

Ethical Compute Utilization (ECU)

While ISR measures suppression efficiency, Ethical Compute Utilization (ECU) evaluates the percentage of inference cycles executed with a valid policy alignment against the total actual executed cycles:

$$\text{Equation: } ECU = \frac{\text{Inference Cycles with Valid } J(t)}{\text{Total Executed Cycles}} \times 100\%$$

Justification triggers include formal schedule phase transitions or basic heuristic prompts. The operational target for the system is an $ECU > 90\%$.

Context-Aware Duty Cycle (CADC)

The operational footprint of each individual module is captured by the Context-Aware Duty Cycle (CADC), defined as the ratio of a module's active time to the total lecture duration:

$$\text{Equation: } CADC = \frac{\text{Module Active Time}}{\text{Lecture Duration}} \times 100\%$$

Under this model, computationally heavy models are restricted to minimal CADC profiles, while lightweight models (such as pose detection) can afford to maintain high CADC values.

FAILURE CONTAINMENT DESIGN

Failure Mode Analysis

Because edge runtimes are inherently susceptible to local component failures, asynchronous health-report lag, and jittery scheduling, the orchestration layer actively contains these effects. Table I specifies the failure scenarios and containment measures that we implement, which are inspired by dependable computing taxonomies [Ref] and common microservice degradations [Ref]. Graceful degradation is pursued as an orchestration design goal, and not an accidental outcome of modularity. The system is engineered to favor operational continuity over sensing completeness: fallback sensing mechanisms are implemented at all levels so that secondary modules can be brought to bear when primary modules fail.

TABLE: Failure Mode Analysis and Mitigation

Failure isolation is achieved through runtime separation combined with serialized context exchange. Under evaluated scenarios, a crash within the Perception layer typically results in localized termination. The Reasoning and Governance layers, running in separate contexts, continue execution and initiate fallback procedures via the health interface.

Abstract State Transitions for Graceful Degradation

The Governance layer relies on deterministic abstract state transitions to track system health. Transitions between the states—NORMAL, DEGRADED, and SAFE—are executed as logical operations based on available health signals.

The abstract logical system gracefully degrades from NORMAL (all modules) to DEGRADED (pose-only) to SAFE (occupancy sensors) based on abstract module health. We do not define the execution semantics or physical state machine implementations here, only the control decisions. The Governance scheduler assumes health signals are natively available from the underlying runtime execution layers.

POLICY-AWARE SCHEDULER

Operational Budgets

To prevent excessive resource drain, the Governance layer tracks daily compute and activation budgets for each module:

Equation:
$$B_{\text{module}}(\text{day}) \leq B_{\text{max}} = 100 \text{ activations/day}$$

When the daily operational budget is exhausted, the system automatically falls back to lower-tier operational modes to conserve energy and compute resources.

Session Boundary Enforcement

At the end of a lecture, the Governance layer actively triggers teardowns to terminate the active perceptual processes and reset the logical context for the subsequent lecture, thus establishing strict boundaries between different pedagogical events.

VALIDATION VIA FAULT INJECTION

While detailed systematic evaluation is a matter for the validation layer [Ref], the control-plane behavior described was stress-tested under fault injection to establish logical isolation. This was found to be the case in the current implementation: abstract state transition proceed deterministically even when lower-level perception modules time out or return memory pressure. Claims about the system’s ability to withstand chaos, convergence time to policy-preserving states after faults, or latency properties are deferred to the validation architecture; for now, sufficiency is established for the control decisions to respect the policy.

EXPERIMENTAL RESULTS

We now examine the operational behavior of the control-plane architecture under simulated lecture workloads. This section evaluates orchestration performance strictly in terms of module activation patterns.

begin{center}

\fbox{\parbox{0.9\columnwidth}{

Evaluation Scope: Metric profiling was performed as part of controlled orchestration test harness. The metrics reflect the control plane logic and activation states, they are not indicative of the overall system performance or end to end task latency.

}}

\end{center}

Inference Suppression Statistics

During a 60-minute simulation lecture, the Compute Justification Model demonstrated a substantial decrease in the number of active cycles. The system registered 72% of the heavy vision modules’ ISR, indicating that most intensive capture was performed during justified time. In addition, the ASR module suppression hit 85%, while the lightweight pose estimator utilization reached 12% ISR.

[FIGURE: Architectural Pipeline / Validation Curves]

\caption{Inference Suppression Ratio (ISR) by module. Heavy Vision suppressed 72% of scheduled cycles; ASR 85%; Pose only 12%.}

\label{fig:isr}

\end{figure}

It is noteworthy that preliminary tuning achieved suppression rates of over 80%, for the heavy vision modules, but at the expense of responsiveness during spontaneous dialogues. The suppression was later reduced to acceptable levels for the dialog module, to preserve interactive responsiveness.

Module Activation Timeline

[FIGURE: Architectural Pipeline / Validation Curves]

}

\caption{Module activation timeline: All modules active (0-10 min), lightweight mode (10-40 min), interactive mode (40-55 min), no modules during dismissal (55-60 min).}

\label{fig:timeline}

\end{figure}

Figure \ref{fig:timeline} shows the variation of the activated modules over the time of the demo lecture. Heavy vision modules were only active for 25 out of the total 60 min, therefore reaching only 42% Context-Aware Duty Cycle (CADC). On the other hand,

the lightweight pose modules provided a constant 75% CAD/C, supplying the geometric context continuously without notable overhead.

Limitations

The orchestration framework exposes several operational limitations:

- **Heuristic-Policy Dependence:** The model relies on defined heuristic schedules; spontaneous classroom interaction shifts or organic deviation may result in unintended sensing suppression.
- **Calibration Sensitivity:** Tuning the Inference Suppression Ratio (ISR) to balance operational savings with interactive responsiveness requires site-specific empirical calibration.
- **Scheduling Rigidity:** Rigid enforcement of lecture session boundaries struggles with overlapping room utilization or ad-hoc interactions.
- **Asynchronous Race-Condition Risk:** Abstract state transitions are validated by fault injection only; unexpected race-conditions might be present in the asynchronous healthy monitoring loops under heavy contention of edge-runtime.
- **Workload Unpredictability:** Heavy vision modules exhibit variable execution times depending on frame complexity, introducing scheduling jitter that synchronous timeout boundaries may misinterpret as module failure.

DISCUSSION

Modifying Policy Constraints Encoded inside the Orchestration Layer The way edge systems allocate their computational resources changes when policy constraints are embedded into the orchestration layer, as it no longer utilizes a brute force approach to constantly capture and process all available signals, but rather only enables certain modules according to institutional constraints [Ref].

However, deployment calibration remains an engineering challenge. Adjusting the thresholds according to different classroom layouts might require some iterative engineering during installations. Adjusting these heuristics is crucial to ensure not wasting computation on false positives and to achieve substantial savings in compute.

Future directions in this line of research include predictive IRG scaling based on machine-learned anticipation of the lecture phase to elastically scale the module before a trigger, as well as multi-classroom control plane orchestration across a federated cluster of edge nodes.

CONCLUSION

This study investigated whether heterogeneous AI modules operating at the edge can be coordinated through a layered control-plane abstraction that preserves policy-aware activation, fault containment, and deterministic degradation. By separating Perception, Reasoning, and Governance responsibilities and restricting cross-layer communication to typed context vectors, the architecture isolates orchestration faults and prevents localized module crashes from causing cascading pipeline failure.

The Compute Justification Model explicitly defines the activation of the given modules according to the defined phases, which proves that orchestration under the given constraints can be deterministic. The analysis of the fault injection in the control plane of the considered orchestration shows that it is possible to minimize the extra activation of the elements by deterring the failure of the critical perception pipelines.

The proposed metrics—ISR, ECU, CAD/C, and LIF—provide a quantitative framework for analyzing activation governance. Within the ScholarMaster architecture, this framework operates exclusively as the runtime-governance orchestration subsystem—managing deterministic module coordination and activation transitions without evaluating survivability envelopes (Paper 10), proposing new federated models (Paper 13), or defining irreversible privacy

architectures (Paper 17).

REFERENCES

- [1] A. Avizienis et al., "Basic Concepts and Taxonomy of Dependable and Secure Computing," IEEE Transactions on Dependable and Secure Computing, vol. 1, no. 1, pp. 11-33, 2004.
- [2] P. Alvaro et al., "Lineage-driven Fault Injection," in Proc. ACM SIGMOD, 2015, pp. 331-346.
- [3] M. Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning," in Proc. OSDI, 2016, pp. 265-283.
- [4] W. Shi et al., "Edge Computing: Vision and Challenges," IEEE Internet of Things Journal, vol. 3, no. 5, pp. 637-646, 2016.
- [5] "Regulation (EU) 2016/679 (General Data Protection Regulation)," Official Journal of the European Union, L119, 2016.
- [6] M. Chiang and T. Zhang, "Fog and IoT: An Overview of Research Opportunities," IEEE Internet of Things Journal, vol. 3, no. 6, pp. 854-864, 2016.
- [7] D. McDuff et al., "AFFDEX SDK: A Cross-Platform Real-Time Multi-Face Expression Recognition Toolkit," in Proc. CHI Extended Abstracts, 2016, pp. 3723-3726. % — FEDERATED LEARNING —
- [8] P. Voigt and A. Von dem Bussche, The EU General Data Protection Regulation (GDPR): A Practical Guide. Springer, 2017. % — CLOUD ORCHESTRATION —
- [9] N. Dragoni et al., "Microservices: Yesterday, Today, and Tomorrow," in Present and Ulterior Software Engineering, Springer, 2017, pp. 195-216.
- [10] Y. Kang et al., "Neurosurgeon: Collaborative Intelligence Between the Cloud and Edge Microcloud," in Proc. ASPLOS, 2017, pp. 615-629.
- [11] B. McMahan et al., "Communication-Efficient Learning of Deep Networks from Decentralized Data," in Proc. AISTATS, 2017, pp. 1273-1282. % — EDGE AI FRAMEWORKS —
- [12] R. Buyya et al., "Manifesto for Future Generation Cloud Computing: Research Directions for the Next Decade," ACM Computing Surveys, vol. 51, no. 5, pp. 1-38, 2018. % — MONOLITHIC FER SYSTEMS —
- [13] A. H. Jiang et al., "Mainstream: Dynamic Stem-Sharing for Multi-Tenant Video Processing," in Proc. USENIX Annual Technical Conference (ATC), 2018, pp. 29-42.
- [14] M. Nardelli et al., "Elastic Stream Processing for Distributed Environments," IEEE Internet Computing, vol. 22, no. 1, pp. 54-58, 2018.
- [15] A. Ignatov et al., "AI Benchmark: Running Deep Neural Networks on Android Smartphones," in Proc. ECCV Workshops, 2018, pp. 288-314.
- [16] Z. Zhou et al., "Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing," Proceedings of the IEEE, vol. 107, no. 8, pp. 1738-1762, 2019. % — ARCHITECTURE & RELIABILITY —
- [17] Y. Gan et al., "An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems," in Proc. ASPLOS, 2019, pp. 3-18.

- [18] E. Li et al., "Edge AI: On-Demand Accelerating Deep Neural Network Inference via Edge Computing," *IEEE Transactions on Wireless Communications*, vol. 18, no. 1, pp. 347-359, 2019.
- [19] S. Zuboff, *The Age of Surveillance Capitalism*. PublicAffairs, 2019.
- [20] J. Verbraeken et al., "A Survey on Distributed Machine Learning," *ACM Computing Surveys*, vol. 53, no. 2, pp. 1-33, 2020. % — EDGE COMPUTING —
- [21] N. Min-Allah and S. Alrashed, "Smart Campus: A Sketch," *Sustainable Cities and Society*, vol. 59, p. 102231, 2020. % — GENUINE REPLACEMENTS (2015-2022) —
- [22] F. Yi et al., "Privid: Practical, Privacy-Preserving Video Analytics Queries," in *Proc. NSDI*, 2022, pp. 287-306.